Министерство науки и высшего образования РФ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Новосибирский государственный технический университет»

Кафедра Автоматизированных систем управления

ЛАБОРАТОРНАЯ РАБОТА № 4

«MLOps»

по дисциплине «Методы анализа данных»
Вариант 2. Сервис фильтрации SMS-спама

Выполнил:

Бикмухаметов Таир Олегович

Группа АВТ-314

Факультет АВТФ

Проверил:

Ганелина Наталья Давидовна

Новосибирск 2026

1 Цель работы и постановка задачи

Цель работы – освоить основы MLOps: учёт и сравнение экспериментов в MLflow, регистрацию версий моделей, развёртывание API с использованием Docker и Docker Compose, проверку работоспособности, обновление и откат модели без повторного обучения.

Индивидуальный вариант 2 – сервис фильтрации SMS-спама. По тексту сообщения необходимо определить класс spam или ham. Начальные алгоритмы – TF-IDF + MultinomialNB и TF-IDF + LogisticRegression. В экспериментах изменяются alpha и C. Основной критерий выбора – F1 для spam; дополнительно сравниваются precision и recall этого класса.

Требуется провести не менее шести экспериментов, сохранить параметры, метрики, теги и артефакты, зарегистрировать две версии одной модели и назначить alias champion. Затем необходимо экспортировать Pipeline, реализовать API, собрать Docker-образы, проверить корректные и некорректные запросы, показать обновление и откат, проверить healthcheck в Compose и зафиксировать материалы для воспроизведения.

2 Описание набора данных и подготовка

Использован SMS Spam Collection из UCI Machine Learning Repository [1]. Авторы – Tiago Almeida и José Hidalgo; DOI: 10.24432/C5CC84. Набор распространяется по лицензии CC BY 4.0. Источник и исходные байты сохранены в проекте.

Таблица 1. Признаки и целевая переменная

Поле	**Описание**
text	Исходный текст SMS; единственный входной признак.
label / target	Целевой класс: ham = 0, spam = 1.
row_id	Номер строки исходного файла; используется для разбиения, в модель не передаётся.
text_hash	SHA-256 нормализованного текста; используется для контроля повторов.

Файл SMSSpamCollection прочитан разделением каждой непустой строки по первой табуляции. Кавычки не интерпретируются как CSV-разметка. Проверены допустимые классы и непустые тексты. Для обнаружения дубликатов выполняются Unicode NFKC-нормализация, casefold и свёртка пробелов. Сохраняется первое сообщение каждой группы. Нормализация для поиска повторов не заменяет входной текст модели; классификатор получает исходную SMS.

Таблица 2. Состав данных до и после очистки

Этап	**Всего**	**ham**	**spam**
Исходный файл	5574	4827	747
После удаления повторов	5159	4517	642

Исключено 415 повторов. Групп с противоречащими метками: 0. Журнал исключений сохранён в data/excluded.csv. Доля spam после очистки составляет 12,44 %. Поэтому accuracy рассматривается как дополнительная метрика: она не отражает полноту обнаружения редкого класса сама по себе.

2.1 Разбиение и предотвращение утечки

Выполнено стратифицированное разбиение приблизительно 60/20/20 с random_state = 42. Идентификаторы исходных строк сохранены в data/splits.json. Множества row_id и хешей сообщений попарно не пересекаются. Сначала удаляются повторы, затем выполняется разбиение: одинаковые сообщения не оказываются одновременно в обучающей и отложенной выборках.

Таблица 3. Состав выборок

Выборка	**Назначение**	**Всего**	**ham**	**spam**
Train	Обучение TF-IDF и классификатора	3095	2710	385
Validation	Выбор гиперпараметров и модели	1032	903	129
Test	Итоговая оценка выбранной модели	1032	904	128

TF-IDF обучается только на train внутри sklearn Pipeline. На validation и test применяются уже обученные преобразования. Test не используется для выбора параметров или порога. После выбора модель не переобучается на

train + validation: сохраняется соответствие оценённого run, Registry-версии и экспортированного Pipeline.

2.2 Метрики и правило выбора

Положительный класс – spam. Precision = TP / (TP + FP), recall = TP / (TP + FN), F1 = 2 × precision × recall / (precision + recall). Precision характеризует долю настоящего спама среди отфильтрованных сообщений, recall – долю найденного спама среди всех spam-SMS. FP означает ошибочное блокирование обычной SMS, FN – пропуск спама.

Правило выбора задано заранее: максимальный F1 для spam на validation; при равенстве – максимальная precision, затем порядок конфигураций в коде. Порог принятия решения фиксирован на 0,5. При точном равенстве вероятностей выбирается ham, как в sklearn predict.

3 Программное окружение и структура проекта

Работа выполнена в Ubuntu 24.04 под WSL 2. Docker Desktop использует Linux engine и интеграцию с дистрибутивом Ubuntu. Обучение и MLflow выполняются на хосте, API – в контейнере. Графический интерфейс MLflow доступен локально на порту 5000, API через Compose – на порту 8000.

Таблица 4. Версии программного окружения

Компонент	**Версия**	**Компонент**	**Версия**
Python, хост	3.12.3	FastAPI	0.115.12
Python, образ	3.12.10	Pydantic	2.11.5
MLflow	2.22.2	Uvicorn	0.34.2
scikit-learn	1.6.1	SQLAlchemy	2.0.44
NumPy	2.2.6	Docker Engine	29.4.3
Pandas	2.2.3	Docker Compose	5.1.4
SciPy	1.15.3		

Прямые зависимости указаны в requirements.in, полный снимок – в requirements.txt, окружение сервиса – в requirements-runtime.txt. SQLAlchemy закреплена на 2.0.44: версия 2.1.8 несовместима с используемым MLflow.

Таблица 5. Основные файлы и каталоги

Путь	**Назначение**
lab4_mlops.ipynb	Выполненный ноутбук с отчётом и результатами.
sms_mlops/	Подготовка данных, обучение, экспорт, API и проверки.
data/raw/	Оригинальный архив UCI, TSV и readme.
data/splits.json	Точные индексы train/validation/test и контрольная сумма данных.
mlflow-data/	SQLite Tracking/Registry и артефакты запусков.
models/v1/, models/v2/	Pipeline и metadata.json конкретных релизов.
results/	Таблица экспериментов, метрики, графики и протоколы.
screenshots/, examples/	Скриншоты реальных страниц и JSON-запросы.
Dockerfile, compose.yaml	Сборка образов и декларативный запуск сервиса.

4 Последовательность выполнения работы

Все команды выполняются в bash из каталога lab4. Перед запуском Docker Desktop запущен, включена интеграция Settings → Resources → WSL Integration → Ubuntu. Повторная установка Docker не требуется. Для Python используется отдельное виртуальное окружение.

```
cd /home/tair/NETI/AI/lab4
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip check
docker version
docker compose version
python -m sms_mlops.train
python -m sms_mlops.verify
python -m pytest -q
python -m sms_mlops.docker_demo --require
```

Обучение записывает результаты непосредственно в SQLite. Для этого не нужен предварительно запущенный HTTP-сервер MLflow. Для просмотра тех же запусков в другом терминале запущен интерфейс:

```
mlflow server --backend-store-uri sqlite:///mlflow-data/mlflow.db \
  --default-artifact-root ./mlflow-data/artifacts --no-serve-artifacts \
  --host 127.0.0.1 --port 5000 --workers 1
```

Открыт адрес http://127.0.0.1:5000. Эксперимент называется lab4-sms-spam. В интерфейсе показаны шесть завершённых запусков с гиперпараметрами и validation-метриками (рис. 1).

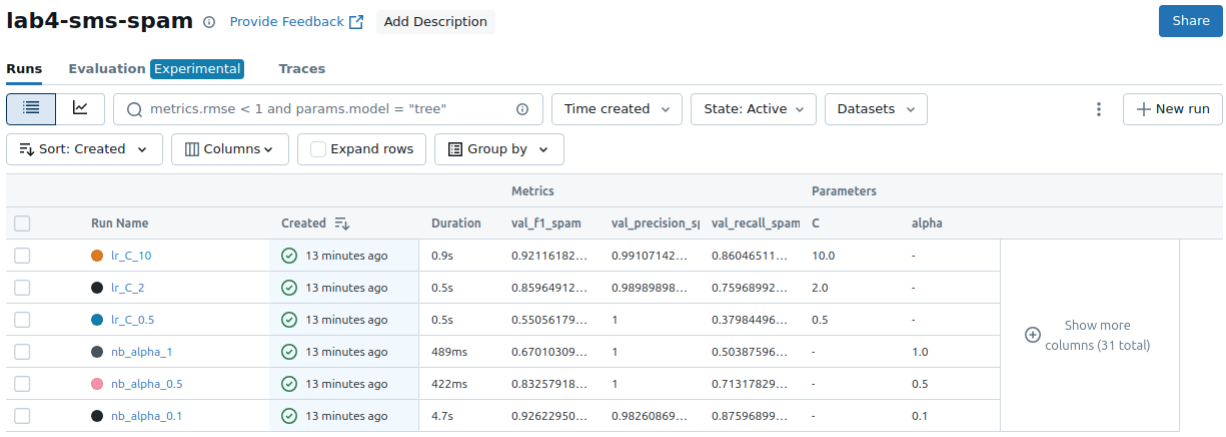


Рис. 1 – Шесть успешных экспериментов в MLflow

При наличии результатов train повторно использует сохранённые запуски. Ключ --force предназначен для нового цикла: он добавляет шесть run и две новые Registry-версии, обновляет экспорты и требует повторной сборки образов. Для чтения готового отчёта повторное обучение не требуется.

5 Сравнение экспериментов

Во всех экспериментах используется word TF-IDF: униграммы и биграммы, max_features = 20 000, min_df = 1, sublinear_tf = True, lowercase = True и L2-нормировка. Для MultinomialNB исследованы alpha = 0,1; 0,5; 1. Для LogisticRegression исследованы C = 0,5; 2; 10, solver = liblinear, max_iter = 2000, random_state = 42. Разбиение одинаково для всех моделей.

Таблица 6. Гиперпараметры, validation-метрики и идентификаторы запусков

№	Тип	Параметр	F1 spam	P spam	R spam	Acc.	run_id
1	NB	alpha = 0,1	0,9262	0,9826	0,8760	0,9826	4f7499ea175841c5 a006088d92de6714
2	NB	alpha = 0,5	0,8326	1,0000	0,7132	0,9641	0e234ea27f964c91 abaf567432322b43
3	NB	alpha = 1,0	0,6701	1,0000	0,5039	0,9380	6b6bb00ba9404370 8a6fdbd8685ad6eb
4	LR	C = 0,5	0,5506	1,0000	0,3798	0,9225	d4f0c1ebfeb34173 9b16ea82e9f902b3
5	LR	C = 2,0	0,8596	0,9899	0,7597	0,9690	0b868d7901f14dd8 8c9bb7dc5362f41f
6	LR	C = 10,0	0,9212	0,9911	0,8605	0,9816	cd9cab5f394d490a 9600eea8e5c0dfd2

NB – MultinomialNB, LR – LogisticRegression; P – precision, R – recall, Acc. – accuracy. Идентификаторы run_id перенесены на две строки без изменения символов. Полная таблица, включая время обучения и размер словаря, сохранена в results/experiments.csv.

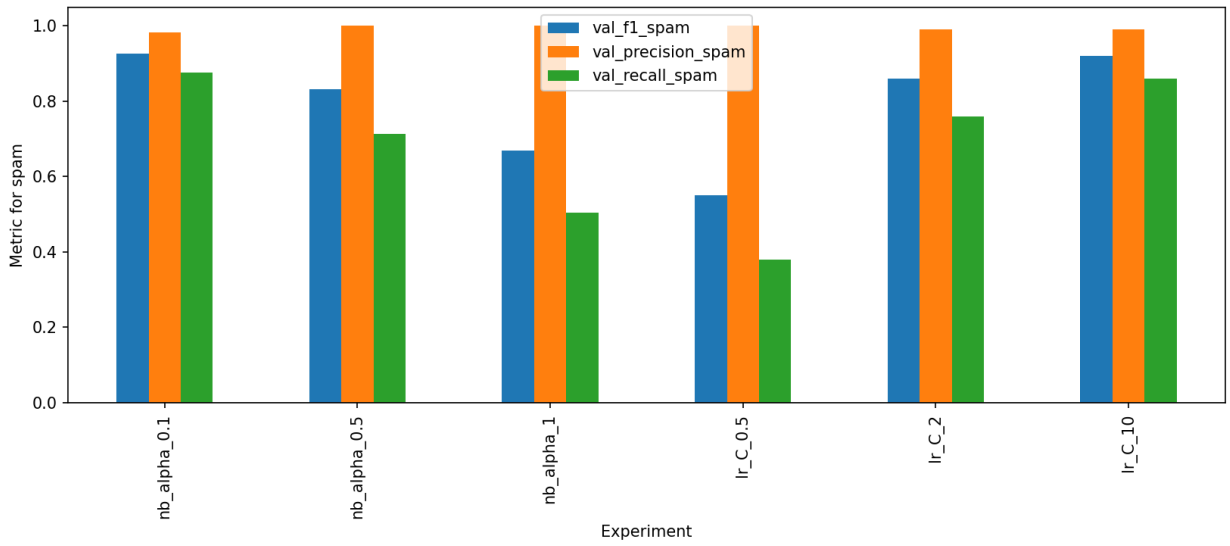


Рис. 2 – Сравнение F1, precision и recall для spam на validation

На рис. 3 приведён скриншот MLflow: видны шесть запусков и validation F1, precision, recall для spam. Настройки alpha и C отражены в названиях запусков и сравнительной таблице. Размер словаря во всех опытах равен 20 000. Числа на графиках MLflow округлены; точные значения приведены в таблице 6.

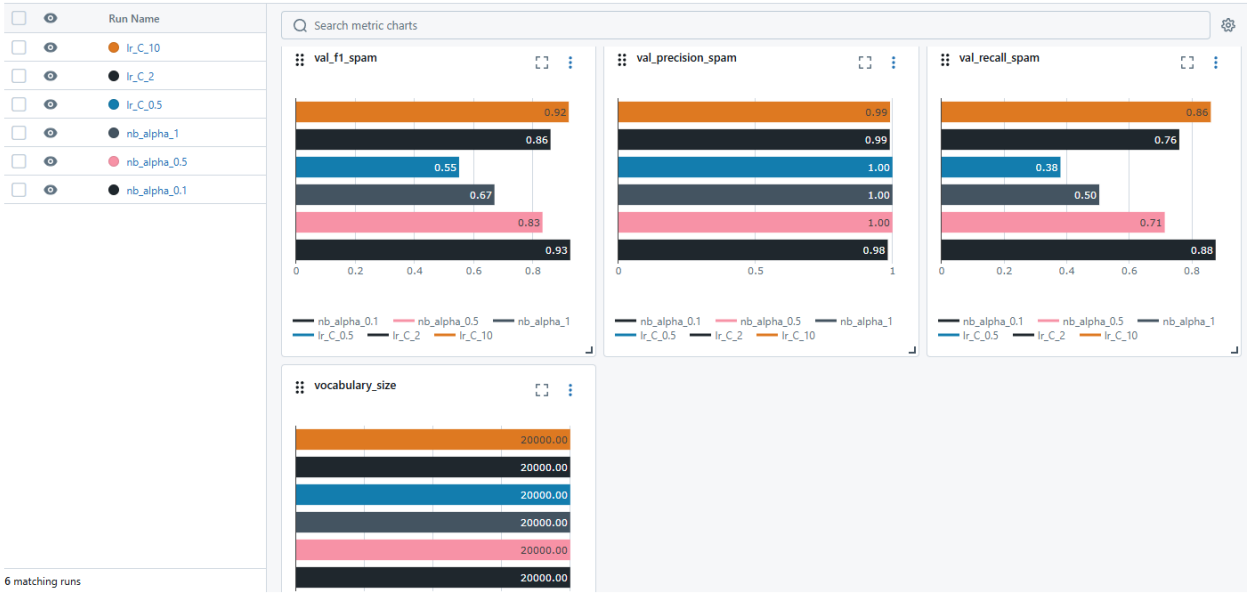


Рис. 3 – Шесть запусков MLflow: validation-метрики и размер словаря

В этой сетке увеличение alpha усиливает сглаживание NB и снижает recall. При alpha = 0,5 и 1 precision равна 1, но часть спама пропускается, поэтому F1 снижается. У LogisticRegression рост C ослабляет регуляризацию и повышает recall в исследованном диапазоне. Эти зависимости наблюдаются на данном разбиении и не гарантируются для любых данных.

В каждом run сохранены параметры модели и TF-IDF, seed, размеры выборок, validation F1/precision/recall/accuracy, время обучения и размер словаря. Теги включают вариант, положительный класс, правило выбора и SHA-256 данных и разбиения. Артефакты содержат Pipeline с signature и input_example, матрицу ошибок, код, зависимости и описание разбиения. Dataset inputs фиксируют роли train и validation. Итоговые test-метрики записаны только выбранному run.

6 Обоснование выбора и итоговая оценка

Выбрана модель MultinomialNB с alpha = 0,1: validation F1 = 0,926230, precision = 0,982609, recall = 0,875969. Лучший представитель другого семейства – LogisticRegression с C = 10, validation F1 = 0,921162. Разница между лидерами невелика; по одному разбиению нельзя утверждать статистически доказанное превосходство.

Таблица 7. Итоговые метрики выбранного Pipeline на test

Метрика	**Значение**
F1 для spam	0,925620
Precision для spam	0,982456
Recall для spam	0,875000
Accuracy	0,982558
Число test-SMS	1032

Скриншоты на рис. 4 и 5 показывают время обучения всех шести моделей, validation accuracy и итоговые test-метрики. На графиках test присутствует только выбранный run nb_alpha_0.1: остальные модели сравнивались на validation, а итоговая оценка test выполнена после выбора.

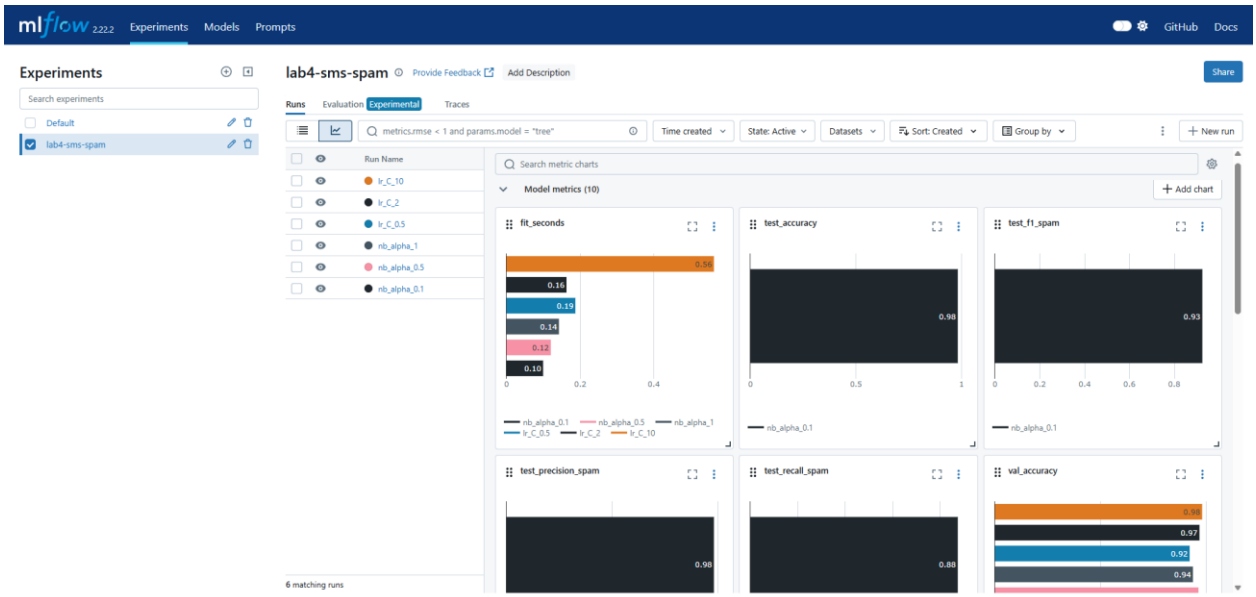


Рис. 4 – Время обучения шести моделей и test accuracy/F1 чемпиона

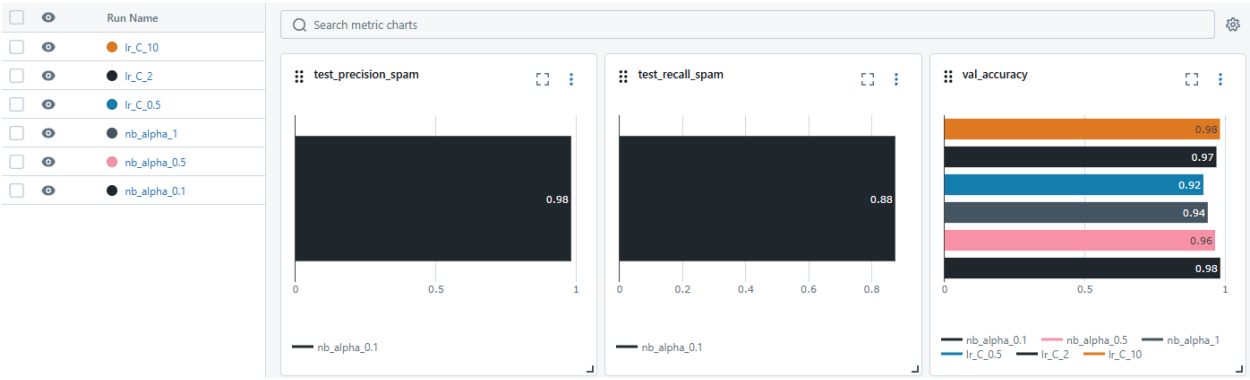


Рис. 5 – Test precision/recall чемпиона и validation accuracy шести моделей

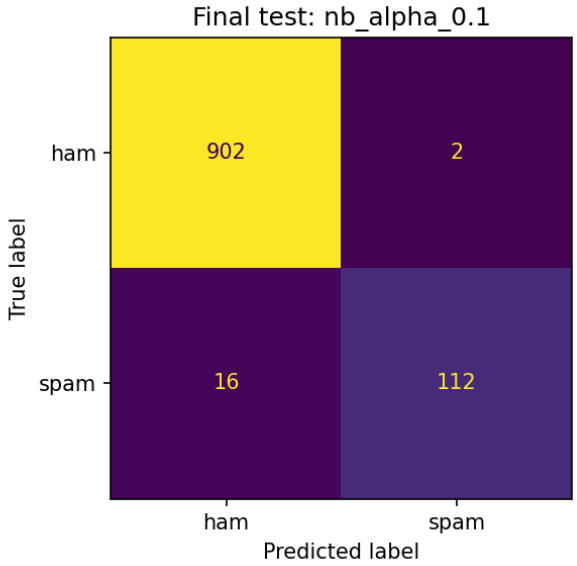


Рис. 6 – Матрица ошибок выбранной модели на test

Правильно обнаружены 112 spam-SMS, пропущены 16; 2 обычных сообщения ошибочно отнесены к spam, 902 сообщения ham распознаны правильно. Строки матрицы соответствуют истинным классам, столбцы – предсказанным, порядок классов: ham, spam. Высокая precision не означает обнаружение всего спама: recall составляет 87,5 %. После получения test-результатов параметры и порог не изменялись.

7 Регистрация версий и alias champion

Под именем sms-spam-filter зарегистрированы два уже обученных Pipeline. Регистрация не создаёт нового обучения. Релиз v1 содержит лучшую LogisticRegression, релиз v2 – выбранный MultinomialNB. Номера Registry получены из ответа MLflow, а не выведены из тегов образов.

Таблица 8. Версии модели в Model Registry

Релиз	Registry	Алгоритм	run_id
v1	1	LogisticRegression	cd9cab5f394d490a 9600eea8e5c0dfd2
v2	2	MultinomialNB	4f7499ea175841c5 a006088d92de6714

Alias champion назначен Registry-версии 2. При экспорте alias разрешается один раз в конкретный номер, после чего загружается models:/sms-spam-filter/<номер>. Смена alias не изменяет содержимое существующего Docker-образа и не переключает работающий контейнер автоматически.

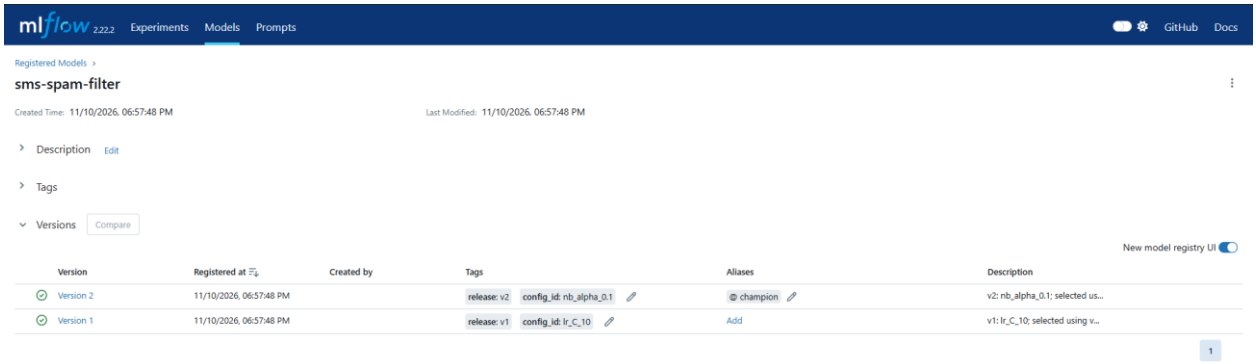


Рис. 7 – Две Registry-версии и alias champion

Экспорт хранится в models/v1 и models/v2. pipeline.joblib содержит и TF-IDF, и классификатор, metadata.json – номер версии, run_id, происхождение, порядок классов, версии окружения и контрольные суммы. Во время предсказания контейнер не обращается к MLflow.

Контрольная сумма Pipeline v1 (SHA-256):

`05026fb8bbed4155e1b3ba8d1ab998765194d9f44fde168625a0adf7e0575c9a`

Контрольная сумма Pipeline v2 (SHA-256):

`c4c5c73d8343999dde42addb99cb0650bbf78be256687014a6f0ea223f4a3d08`

8 API и результаты проверки запросов

Таблица 9. Методы API

Метод и путь	Назначение
GET /health	Готовность загруженной модели, версия и релиз.
GET /metadata	Происхождение экспорта, run_id и SHA-256.
POST /predict	Предсказания для пакета текстов.
GET /docs	Интерактивная документация Swagger UI.

Формат входа: объект с единственным полем texts. Допускается 1–100 строгих строк, каждая длиной 1–5000 символов и не только из пробелов. Числа, логические значения, null, пустые пакеты, лишние поля и повреждённый JSON возвращают 422. Unicode и неизвестные токены допустимы, но качество на русскоязычных SMS отдельно не оценивалось.

```
{"texts": ["See you soon", "Claim your free prize now!"]}
```

Ответ содержит predictions (ham/spam), probabilities_spam, model_version, release, run_id и latency_ms. Вероятности не проходили дополнительную калибровку. Модель загружается один раз при старте. До загрузки проверяется SHA-256 экспорта и порядок классов [0, 1]. Повреждённый файл предотвращает запуск сервиса.

Фактический ответ на сохранённый пакет examples/valid.json:

```
{
  "predictions": [
    "ham",
    "ham",
    "ham"
  ],
  "probabilities_spam": [
    0.0010209079134395204,
    0.1632287445930936,
    0.4945835957774673
  ],
  "model_version": "2",
  "release": "v2",
  "run_id": "4f7499ea175841c5a006088d92de6714",
  "latency_ms": 2.0172039999124536
}
```

Корректность запроса означает соответствие контракту, а не обязательное совпадение каждой метки с истиной. В smoke-примере возможен пропуск спама; качество оценивается отдельно по истинным меткам test. Третий текст примера выбран из validation, где версии дают

разные ответы, чтобы изменение поведения при обновлении было наблюдаемым.

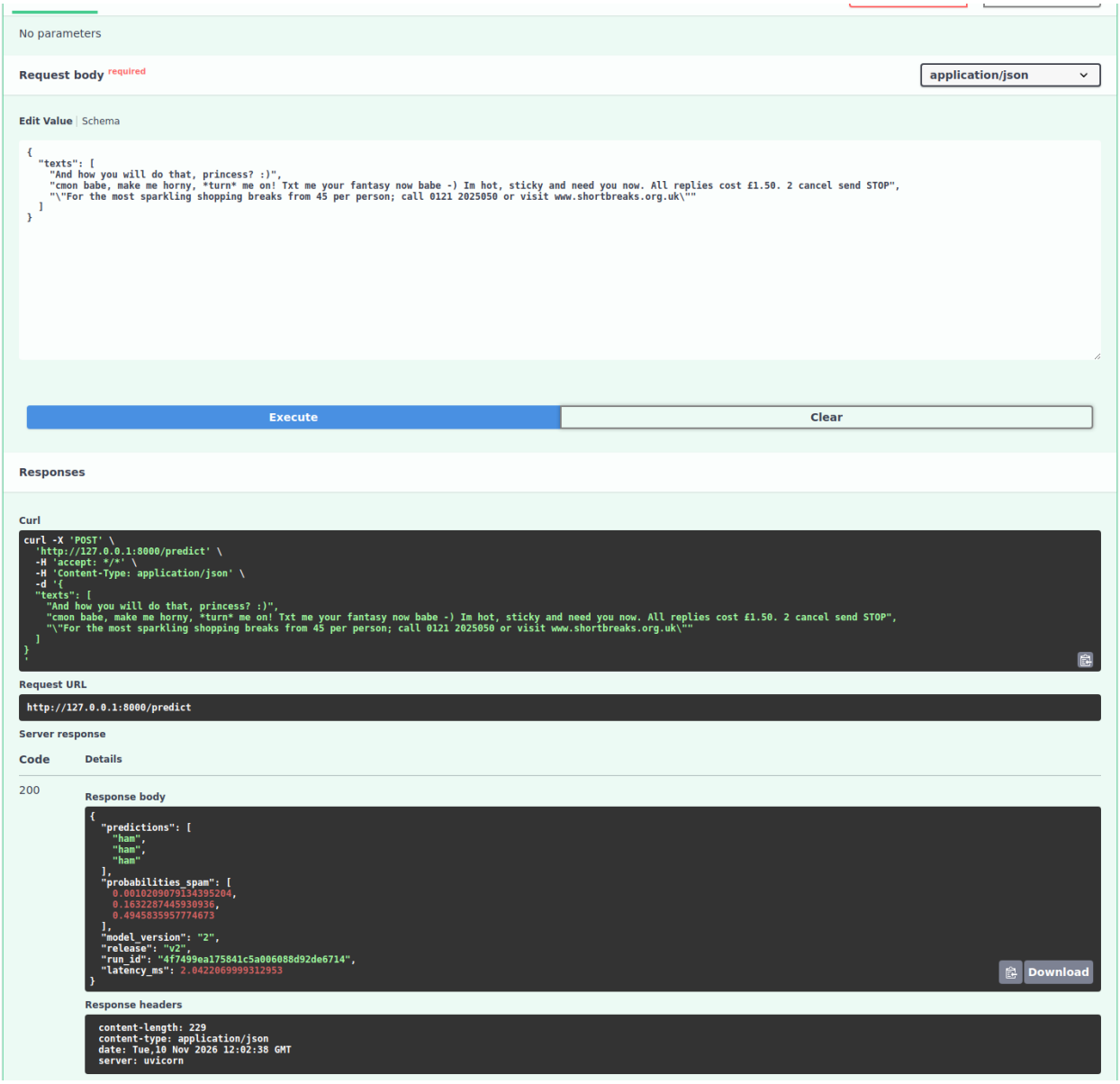


Рис. 8 – Корректный POST /predict через Swagger: запрос и ответ 200

Таблица 10. Некорректные запросы

Случай	**Ожидаемый HTTP-код**	**Полученный код**
Нет texts; неверный тип верхнего уровня	422	422
Пустой пакет; null вместо пакета	422	422
Пустая строка; только пробелы	422	422
Число, boolean или null вместо SMS	422	422
Смешанные типы в пакете	422	422
Строка вместо массива texts	422	422
Лишнее поле в объекте	422	422
101 SMS; сообщение длиннее 5000 символов	422	422
Повреждённый JSON	422	422

Всего проверено 15 разновидностей невалидного ввода. Дополнительно проверены один объект, пакет ровно из 100 строк, Unicode и строка предельной длины. Все проверки выполняются как локально через FastAPI TestClient, так и по HTTP в контейнере.



Рис. 9 – Пустой пакет: контролируемый ответ HTTP 422

```
curl -sS http://127.0.0.1:8000/health
curl -sS -X POST http://127.0.0.1:8000/predict \
  -H 'Content-Type: application/json' --data-binary @examples/valid.json
curl -i -X POST http://127.0.0.1:8000/predict \
  -H 'Content-Type: application/json' --data-binary @examples/invalid_empty.json
```

9 Сборка Docker и проверка совпадения предсказаний

Dockerfile использует python:3.12.10-slim-bookworm, дополнительно закреплённый по digest. Устанавливаются только runtime-зависимости, копируются API и выбранный экспорт. Контейнер работает от appuser с UID 10001. Uvicorn слушает 0.0.0.0:8000 внутри контейнера; публикация на хосте ограничена 127.0.0.1. .dockerignore исключает SQLite, датасет, ноутбук и виртуальное окружение.

```
docker build --build-arg MODEL_RELEASE=v1 -t sms-spam:v1 .
docker build --build-arg MODEL_RELEASE=v2 -t sms-spam:v2 .
docker run --rm -d --name lab4-sms-demo \
  -p 127.0.0.1:18000:8000 sms-spam:v1
python -m sms_mlops.verify --url http://127.0.0.1:18000 --release v1
docker stop lab4-sms-demo
```

Таблица 11. Собранные Docker-образы

Тег	**Размер, MiB**	**Image ID (SHA-256)**
sms-spam:v1	128,82	6d7ceff1a14600064d4fb0c803f8cb94 6acbb80688eaf8ae2a25104d954f7d4e
sms-spam:v2	128,96	7b7ed92ef13698f3cf6e985cf5448257 c70f54b71a61b25e1eeb897bb016d26d

Размер взят из docker image inspect и не равен объёму сетевой передачи. Image ID/digest идентифицируют содержимое, тогда как тег можно перезаписать. Полный stdout/stderr сборок сохранён в results/docker_build_v1.txt, results/docker_build_v2.txt и Docker-протоколе.

Таблица 12. Совпадение API с экспортированным Pipeline

Проверка	**Версия**	**Строк**	**Макс. разность вероятностей**	**Результат**
Контейнер, шаг 1	1	1037	0,000000000000	passed
Контейнер, шаг 2	2	1037	0,000000000000	passed
Контейнер, шаг 3	1	1037	0,000000000000	passed

Для каждой версии проверены все 1032 test-SMS и пять дополнительных строк: всего 1037. Метки API точно совпадают с Pipeline.predict, вероятности проверяются np.testing.assert_allclose с rtol = 1e-10 и atol = 1e-12. Эта проверка подтверждает корректность доставки, а не заменяет сравнение с истинными метками.

10 Обновление и откат без повторного обучения

Выполнена последовательность sms-spam:v1 → sms-spam:v2 → sms-spam:v1. Перед каждым запуском alias champion переключается на соответствующую Registry-версию. После готовности API проверяются metadata, версия, предсказания и ошибочные запросы. При откате используется сохранённый образ v1: переобучение, повторный экспорт и пересборка не выполняются.

Таблица 13. Фактические шаги обновления и отката

Шаг	Тег образа	Registry / champion	Метки трёх SMS	Результат
1	sms-spam:v1	1	ham, ham, spam	passed
2	sms-spam:v2	2	ham, ham, ham	passed
3	sms-spam:v1	1	ham, ham, spam	passed

Число запусков обучения до демонстрации – 6, после – 6. SHA-256 обоих pipeline.joblib до и после совпадают. Повторное появление исходных ответов подтверждает возврат предыдущей версии. После демонстрации champion возвращён выбранной по validation версии v2; эта же версия запущена через Compose.

```
docker run --rm -d --name lab4-sms-demo \
  -p 127.0.0.1:18000:8000 sms-spam:v2
python -m sms_mlops.verify --url http://127.0.0.1:18000 --release v2
docker stop lab4-sms-demo
docker run --rm -d --name lab4-sms-demo \
  -p 127.0.0.1:18000:8000 sms-spam:v1
python -m sms_mlops.verify --url http://127.0.0.1:18000 --release v1
docker stop lab4-sms-demo
```

Таблица 14. Наблюдаемая длительность ответа, пакет из трёх SMS

Шаг / релиз	Медиана HTTP, мс	p95 HTTP, мс	Медиана обработчика, мс
1 / v1	2,316	2,964	0,718
2 / v2	2,475	3,163	0,871
3 / v1	2,724	2,926	0,819

На каждом шаге выполнено десять последовательных запросов. Время обработчика измеряет вычисление, HTTP-время включает транспорт и сериализацию. Это smoke-измерение, а не нагрузочный тест. Обновление с остановкой контейнера допускает краткую недоступность.

11 Docker Compose и healthcheck

Compose использует уже собранный sms-spam:v2. Healthcheck каждые 5 секунд запрашивает /health внутри контейнера, timeout – 3 секунды, start_period – 10 секунд, retries – 5. Команда --wait ожидает готовности сервиса. restart: unless-stopped перезапускает завершившийся контейнер; само состояние unhealthy не вызывает автоматического перезапуска.

```
MODEL_RELEASE=v2 docker compose up -d --no-build --wait --wait-timeout 90
docker compose ps
docker inspect --format '{{.State.Health.Status}}' "$(docker compose ps -q api)"
curl -f http://127.0.0.1:8000/health
```

Фактический результат проверки состояния:

```
healthy
```

Сводка docker compose ps:

Таблица 15. Состояние Compose-сервиса

Параметр	**Значение**
Service	api
Image	sms-spam:v2
State	running
Health	healthy
Проверка API	passed
Registry version	2

Pretty-print

{"status":"ok","model_version":"2","release":"v2"}

Рис. 10 – Ответ /health контейнерного сервиса

Начало зафиксированного контейнерного прогона: 10.11.2026 19:13:17 (Новосибирск, UTC+7). По HTTP дополнительно повторены все проверки контракта и совпадения с экспортом. Остановка сервиса после работы: docker compose down.

12 Выводы по работе

Создан воспроизводимый сервис фильтрации SMS-спама: сохранены исходные данные UCI, решения об исключении повторов, индексы разбиения, seed, зависимости, MLflow-запуски, Registry и экспортированные Pipeline. Полный Pipeline сохраняет обученный TF-IDF вместе с классификатором, что предотвращает расхождение преобразований при доставке.

Сравнены шесть конфигураций двух алгоритмов. По validation F1 выбрана MultinomialNB с alpha = 0,1. На отложенном test получены F1 = 0,9256, precision = 0,9825, recall = 0,8750. Анализ precision/recall показал, что при высокой точности положительных ответов сохраняются пропуски спама. Увеличение alpha снижало recall NB, увеличение C повышало recall LogisticRegression в исследованной сетке.

Зарегистрированы две версии одной модели и назначен alias champion. Реальные контейнерные проверки подтвердили совпадение API с экспортом на 1037 текстах, контролируемые ответы 422, обновление и откат без переобучения. Compose завершил запуск со статусом healthy. Размер образов и наблюдаемая длительность ответа сохранены в протоколах.

Ограничения результата: исторический преимущественно англоязычный корпус, одно случайное разбиение, возможные близкие шаблоны после удаления повторов и отсутствие отдельной калибровки вероятностей. Полученная оценка не доказывает такое же качество на современных русскоязычных SMS. Дальнейшее развитие может включать кросс-валидацию на train, отдельный современный корпус и мониторинг изменения данных.

13 Список использованных источников

1. Almeida T., Hidalgo J. SMS Spam Collection. UCI Machine Learning Repository, 2011. DOI: 10.24432/C5CC84. URL: https://archive.ics.uci.edu/dataset/228/sms+spam+collection (дата обращения: 10.11.2026).

2. Лабораторная работа 4. MLOps: методические материалы, PDF, предоставленный к заданию. Разделы 7–17 и индивидуальный вариант 2.

3. MLflow. Model Registry; sklearn API. URL: https://mlflow.org/docs/latest/model-registry/ ; https://mlflow.org/docs/latest/api_reference/python_api/mlflow.sklearn.html (дата обращения: 10.11.2026).

4. scikit-learn 1.6. Pipelines and composite estimators; Text feature extraction; Model evaluation. URL: https://scikit-learn.org/1.6/modules/compose.html ; https://scikit-learn.org/1.6/modules/feature_extraction.html ; https://scikit-learn.org/1.6/modules/model_evaluation.html (дата обращения: 10.11.2026).

5. Docker. Dockerfile reference; Compose services; WSL integration. URL: https://docs.docker.com/reference/dockerfile/ ; https://docs.docker.com/reference/compose-file/services/ ; https://docs.docker.com/desktop/features/wsl/ (дата обращения: 10.11.2026).

6. FastAPI. Request Body. URL: https://fastapi.tiangolo.com/tutorial/body/ (дата обращения: 10.11.2026).

7. Pydantic. Validators. URL: https://docs.pydantic.dev/latest/concepts/validators/ (дата обращения: 10.11.2026).

14 Приложение. Исходный код и конфигурация

Приведены исходные файлы вычислительной части, API, контейнерного сценария и проверок, зависимости, Dockerfile, Compose и примеры запросов. Вместе с отчётом сохраняется вся папка lab4. SQLite, бинарные Pipeline, датасет, изображения и полные протоколы представлены отдельными файлами. Генераторы ноутбука и Word-отчёта находятся в scripts/.

sms_mlops/__init__.py

```
"""Лабораторная работа 4, вариант 2: воспроизводимый SMS spam classifier."""
```

sms_mlops/common.py

```
"""Paths and serialization shared by training and verification."""
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
MODEL_NAME = "sms-spam-filter"
SEED = 42
SOURCE = "https://archive.ics.uci.edu/dataset/228/sms+spam+collection"
DATA_URL = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip"


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def write_json(path, value):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n", encoding="utf-
8")


def read_json(path):
    return json.loads(Path(path).read_text(encoding="utf-8"))


def utc_now():
    return datetime.now(timezone.utc).isoformat()
```

sms_mlops/data.py

```
"""Download once; retain raw bytes, deduplication decisions and split indices."""
import hashlib
import io
import re
import unicodedata
import urllib.request
import zipfile

import pandas as pd
```

```
from sklearn.model_selection import train_test_split

from .common import DATA_URL, ROOT, SEED, SOURCE, read_json, sha256, write_json


def normalized(text):
    return re.sub(r"\s+", " ", unicodedata.normalize("NFKC", text).casefold()).strip()


def prepare_data():
    raw = ROOT / "data/raw/SMSSpamCollection"
    raw.parent.mkdir(parents=True, exist_ok=True)
    if not raw.exists():
        # No implicit download when the committed source file is already present.
        with urllib.request.urlopen(DATA_URL, timeout=60) as response:
            archive_bytes = response.read()
        (raw.parent / "sms_spam_collection.zip").write_bytes(archive_bytes)
        with zipfile.ZipFile(io.BytesIO(archive_bytes)) as archive:
            for name in ("SMSSpamCollection", "readme"):
                (raw.parent / name).write_bytes(archive.read(name))
    # split on the first TAB only: SMS may contain quotes and additional TABs.
    rows = [line.split("\t", 1) for line in raw.read_text(encoding="utf-8").splitlines() if
line]
    if any(len(row) != 2 for row in rows):
        raise ValueError("Invalid source TSV row")
    df = pd.DataFrame(rows, columns=["label", "text"])
    df.index.name = "row_id"
    assert df.label.isin(["ham", "spam"]).all()
    assert df.text.str.strip().ne("").all()
    df["text_hash"] = df.text.map(lambda text:
hashlib.sha256(normalized(text).encode()).hexdigest())
    conflicts = df.groupby("text_hash").label.nunique()
    conflicts = set(conflicts[conflicts > 1].index)
    clean = df[~df.text_hash.isin(conflicts)].drop_duplicates("text_hash").copy()
    clean["target"] = clean.label.map({"ham": 0, "spam": 1})
    split_path = ROOT / "data/splits.json"
    expected_hash = sha256(raw)
    if split_path.exists():
        split = read_json(split_path)
        if split["raw_sha256"] != expected_hash or split["seed"] != SEED:
            raise ValueError("Source/seed changed; saved split must be reviewed
explicitly")
    else:
        train_ids, rest = train_test_split(
            clean.index.to_numpy(), test_size=0.4, random_state=SEED, stratify=clean.target
        )
        val_ids, test_ids = train_test_split(
            rest, test_size=0.5, random_state=SEED, stratify=clean.loc[rest, "target"]
        )
        split = {"seed": SEED, "raw_sha256": expected_hash,
                 "method": "normalized-text deduplication, stratified 60/20/20",
                 "train": train_ids.tolist(), "validation": val_ids.tolist(), "test":
test_ids.tolist()}
        write_json(split_path, split)
    sets = [set(split[name]) for name in ("train", "validation", "test")]
    assert not (sets[0] & sets[1] or sets[0] & sets[2] or sets[1] & sets[2])
    assert set.union(*sets) == set(clean.index)
    frames = {name: clean.loc[split[name]].copy() for name in ("train", "validation",
"test")}
    hashes = [set(frame.text_hash) for frame in frames.values()]
    assert not (hashes[0] & hashes[1] or hashes[0] & hashes[2] or hashes[1] & hashes[2])
    clean.to_csv(ROOT / "data/clean.csv", index=True)
    # This file explains every omitted source row, including conflicting labels.
    excluded = df.loc[~df.index.isin(clean.index)].copy()
    excluded["reason"] = excluded.text_hash.map(
        lambda value: "conflicting labels" if value in conflicts else "duplicate normalized
text"
    )
    excluded.to_csv(ROOT / "data/excluded.csv", index=True)
    summary = {"source": SOURCE, "download_url": DATA_URL, "license": "CC BY 4.0",
               "raw_sha256": expected_hash, "raw_rows": len(df), "clean_rows": len(clean),
               "excluded_rows": len(excluded), "conflicting_hashes": len(conflicts),
               "raw_classes": df.label.value_counts().to_dict(),
               "clean_classes": clean.label.value_counts().to_dict(),
               "split_sha256": sha256(split_path),
```

```
                "splits": {name: {"rows": len(frame),
**frame.label.value_counts().to_dict()}
                           for name, frame in frames.items()}}
    write_json(ROOT / "data/manifest.json", summary)
    return frames, summary


if __name__ == "__main__":
    _, summary = prepare_data()
    print(summary)
```

sms_mlops/docker_demo.py

```
"""Build two releases, run/update/rollback without training, then Compose healthcheck."""
import argparse
import os
import subprocess
import time

import httpx

from .common import MODEL_NAME, ROOT, read_json, sha256, utc_now, write_json
from .train import setup_tracking
from .verify import check_client


def run(require=False, no_build=False):
    report_path = ROOT / "results/docker_checks.json"
    report = {"status": "running", "created_at": utc_now(), "commands": [], "sequence": []}
    write_json(report_path, report)

    def command(args, env=None, timeout=600):
        started = time.perf_counter()
        result = subprocess.run(args, cwd=ROOT, env=env, text=True, capture_output=True,
timeout=timeout)
        entry = {"argv": args, "returncode": result.returncode,
                 "seconds": time.perf_counter() - started, "stdout": result.stdout,
"stderr": result.stderr}
        report["commands"].append(entry)
        write_json(report_path, report)
        if result.returncode:
            raise RuntimeError(f"Command failed: {args}\n{result.stderr or result.stdout}")
        return result.stdout.strip()

    try:
        report["docker_version"] = command(["docker", "version"], timeout=30)
        report["compose_version"] = command(["docker", "compose", "version"], timeout=30)
    except (OSError, RuntimeError, subprocess.TimeoutExpired) as error:
        report.update(status="blocked", reason=str(error),
                      instruction="Start Docker Desktop and enable Resources > WSL
Integration > Ubuntu, then rerun this module.")
        write_json(report_path, report)
        print("Docker недоступен:", str(error), flush=True)
        if require:
            raise
        return report
    client, experiment_id = setup_tracking()
    summary = read_json(ROOT / "results/summary.json")
    before_runs = len(client.search_runs([experiment_id], max_results=50000))
    before_hashes = {r: sha256(ROOT / "models" / r / "pipeline.joblib") for r in ("v1",
"v2")}
    created = False
    container = "lab4-sms-demo"
    try:
        # Refuse to delete any pre-existing container with the same name.
        occupied = command(["docker", "ps", "-aq", "--filter", f"name=^/{container}$"])
        if occupied:
            raise RuntimeError(f"Container {container} already exists. Choose another name
or remove it yourself.")
        report["images"] = {}
        for release in ("v1", "v2"):
            if not no_build:
```

```
                command(["docker", "build", "--build-arg", f"MODEL_RELEASE={release}", "-
t", f"sms-spam:{release}", "."])
                entry = report["commands"][-1]
                (ROOT / "results" /
f"docker_build_{release}.txt").write_text(entry["stdout"] + entry["stderr"])
            import json
            image = json.loads(command(["docker", "image", "inspect", f"sms-
spam:{release}"]))[0]
            report["images"][release] = {"tag": f"sms-spam:{release}", "id": image["Id"],
                                         "repo_digests": image.get("RepoDigests", []),
"size_bytes": image["Size"]}
        for release in ("v1", "v2", "v1"):
            version = summary["releases"][release]["model_version"]
            client.set_registered_model_alias(MODEL_NAME, "champion", version)
            assert str(client.get_model_version_by_alias(MODEL_NAME, "champion").version)
== str(version)
            command(["docker", "run", "--rm", "-d", "--name", container,
                     "-p", "127.0.0.1:18000:8000", f"sms-spam:{release}"])
            created = True
            with httpx.Client(base_url="http://127.0.0.1:18000", timeout=5,
trust_env=False) as http:
                deadline = time.monotonic() + 90
                while True:
                    try:
                        health = http.get("/health")
                        if health.status_code == 200:
                            break
                    except httpx.HTTPError:
                        pass
                    if time.monotonic() > deadline:
                        raise RuntimeError("Container did not become ready")
                    time.sleep(0.5)
                checked = check_client(http, release)
                checked["image_tag"] = f"sms-spam:{release}"
                checked["image_id"] = report["images"][release]["id"]
                checked["champion_alias"] = version
                report["sequence"].append(checked)
            command(["docker", "logs", container])
            command(["docker", "stop", container])
            created = False
        assert report["sequence"][0]["example_response"]["predictions"] ==
report["sequence"][2]["example_response"]["predictions"]
        # After the rollback demonstration restore the validation champion, then serve it
via Compose.
        client.set_registered_model_alias(MODEL_NAME, "champion",
summary["releases"]["v2"]["model_version"])
        environment = {**os.environ, "MODEL_RELEASE": "v2"}
        command(["docker", "compose", "up", "-d", "--no-build", "--wait", "--wait-timeout",
"90"], env=environment)
        report["compose_ps"] = command(["docker", "compose", "ps", "--format", "json"],
env=environment)
        cid = command(["docker", "compose", "ps", "-q", "api"], env=environment)
        report["compose_health"] = command(["docker", "inspect", "--format",
"{{.State.Health.Status}}", cid])
        assert report["compose_health"] == "healthy"
        with httpx.Client(base_url="http://127.0.0.1:8000", timeout=30, trust_env=False) as
http:
            report["compose_checks"] = check_client(http, "v2")
        after_hashes = {r: sha256(ROOT / "models" / r / "pipeline.joblib") for r in ("v1",
"v2")}
        after_runs = len(client.search_runs([experiment_id], max_results=50000))
        assert before_runs == after_runs and before_hashes == after_hashes
        report["no_retraining"] = {"runs_before": before_runs, "runs_after": after_runs,
                                   "hashes_before": before_hashes, "hashes_after":
after_hashes}
        report["status"] = "passed"
    except Exception as error:
        report.update(status="failed", reason=repr(error))
        raise
    finally:
        if created:
            command(["docker", "stop", container])
        client.set_registered_model_alias(MODEL_NAME, "champion",
summary["releases"]["v2"]["model_version"])
        write_json(report_path, report)
```

```
    print("PASS: Docker v1 → v2 → v1, Compose healthy; no retraining", flush=True)
    return report


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--require", action="store_true", help="Fail if Docker is
unavailable")
    parser.add_argument("--no-build", action="store_true", help="Use already built v1/v2
images and verify them against exports")
    args = parser.parse_args()
    run(require=args.require, no_build=args.no_build)
```

sms_mlops/relocate.py

```
"""Rebase MLflow local artifact paths when the whole lab is moved to another folder."""
import sqlite3
from pathlib import Path

from .common import ROOT


def relocate_store(root=ROOT):
    root = Path(root).resolve()
    database = root / "mlflow-data/mlflow.db"
    if not database.exists():
        return 0
    new_artifacts = str(root / "mlflow-data/artifacts")
    # SQL identifiers are fixed, values are always bound parameters.
    fields = [("experiments", "artifact_location"), ("runs", "artifact_uri"),
              ("model_versions", "source"), ("model_versions", "storage_location")]
    changes = []
    with sqlite3.connect(database) as connection:
        for table, column in fields:
            exists = connection.execute("SELECT name FROM sqlite_master WHERE type='table'
AND name=?", (table,)).fetchone()
            if not exists:
                continue
            for (old,) in connection.execute(f"SELECT DISTINCT {column} FROM {table}"):
                if not old or "/mlflow-data/artifacts" not in old:
                    continue
                old_base, tail = old.split("/mlflow-data/artifacts", 1)
                prefix = "file://" if old_base.startswith("file://") else ""
                new = prefix + new_artifacts + tail
                if old != new:
                    changes.append((table, column, old, new))
        if changes:
            backup_path = database.with_name("mlflow.pre-relocation.db.bak")
            with sqlite3.connect(backup_path) as backup:
                connection.backup(backup)
            for table, column, old, new in changes:
                connection.execute(f"UPDATE {table} SET {column}=? WHERE {column}=?", (new,
old))
    return len(changes)


if __name__ == "__main__":
    print("Rebased artifact paths:", relocate_store())
```

sms_mlops/service.py

```
"""FastAPI inference; loading never imports MLflow or contacts the Registry."""
import hashlib
import json
import os
import time
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Annotated
```

```
import joblib
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, Field, StringConstraints, field_validator

Text = Annotated[str, StringConstraints(strict=True, min_length=1, max_length=5000)]


class PredictRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", strict=True)
    texts: list[Text] = Field(min_length=1, max_length=100)

    @field_validator("texts")
    @classmethod
    def reject_blank(cls, texts):
        if any(not text.strip() for text in texts):
            raise ValueError("SMS must contain at least one non-whitespace character")
        return texts


class PredictResponse(BaseModel):
    predictions: list[str]
    probabilities_spam: list[float]
    model_version: str
    release: str
    run_id: str
    latency_ms: float


def create_app(model_dir=None):
    folder = Path(model_dir or os.environ.get("MODEL_DIR", "models/v2")).resolve()

    @asynccontextmanager
    async def lifespan(app):
        metadata = json.loads((folder / "metadata.json").read_text())
        pipeline_path = folder / "pipeline.joblib"
        digest = hashlib.sha256(pipeline_path.read_bytes()).hexdigest()
        if digest != metadata["pipeline_sha256"]:
            raise RuntimeError("Exported model SHA-256 mismatch")
        pipeline = joblib.load(pipeline_path)
        if pipeline.classes_.tolist() != [0, 1]:
            raise RuntimeError("Expected ordered classes [ham=0, spam=1]")
        app.state.pipeline = pipeline
        app.state.metadata = metadata
        yield

    app = FastAPI(title="SMS spam filter — вариант 2", version="1.0.0", lifespan=lifespan)

    @app.get("/health")
    def health():
        return {"status": "ok", "model_version": app.state.metadata["model_version"],
                "release": app.state.metadata["release"]}

    @app.get("/metadata")
    def metadata():
        return app.state.metadata

    @app.post("/predict", response_model=PredictResponse)
    def predict(body: PredictRequest):
        started = time.perf_counter()
        probabilities = app.state.pipeline.predict_proba(body.texts)[:, 1]
        # Same threshold and tie rule as sklearn's predict (class 0 wins a tie).
        predicted = ["spam" if probability > 0.5 else "ham" for probability in
probabilities]
        meta = app.state.metadata
        return {"predictions": predicted, "probabilities_spam": probabilities.tolist(),
                "model_version": meta["model_version"], "release": meta["release"],
                "run_id": meta["run_id"], "latency_ms": (time.perf_counter() - started) *
1000}

    return app


app = create_app()
```

sms_mlops/train.py

```
"""Six validation experiments, Registry versions, immutable exports, final test."""
import argparse
import importlib.metadata
import os
import platform
import sys
import time

os.environ.setdefault("MPLCONFIGDIR",
str(__import__("pathlib").Path(__file__).resolve().parents[1] / ".runtime/matplotlib"))

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import mlflow
import mlflow.sklearn
import numpy as np
import pandas as pd
from mlflow import MlflowClient
from mlflow.models import infer_signature
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score, classification_report,
                             confusion_matrix, f1_score, precision_score, recall_score)
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

from .common import MODEL_NAME, ROOT, SEED, SOURCE, read_json, sha256, utc_now, write_json
from .data import prepare_data
from .relocate import relocate_store

CONFIGS = [("nb", "alpha", value) for value in (0.1, 0.5, 1.0)] + [
    ("lr", "C", value) for value in (0.5, 2.0, 10.0)]


def setup_tracking():
    # No external server is needed to train. The UI opens this same SQLite store.
    folder = ROOT / "mlflow-data"
    folder.mkdir(exist_ok=True)
    relocate_store()
    mlflow.set_tracking_uri(f"sqlite:///{folder / 'mlflow.db'}")
    mlflow.set_registry_uri(mlflow.get_tracking_uri())
    client = MlflowClient()
    exp = client.get_experiment_by_name("lab4-sms-spam")
    if exp is None:
        # Relative artifact root keeps the whole lab movable. Always run UI from lab4.
        experiment_id = client.create_experiment("lab4-sms-spam",
artifact_location="./mlflow-data/artifacts")
    else:
        experiment_id = exp.experiment_id
    mlflow.set_experiment(experiment_id=experiment_id)
    return client, experiment_id


def metrics(y_true, predicted, prefix):
    return {f"{prefix}_f1_spam": f1_score(y_true, predicted, pos_label=1, zero_division=0),
            f"{prefix}_precision_spam": precision_score(y_true, predicted, pos_label=1,
zero_division=0),
            f"{prefix}_recall_spam": recall_score(y_true, predicted, pos_label=1,
zero_division=0),
            f"{prefix}_accuracy": accuracy_score(y_true, predicted)}


def export_version(client, version, release, dataset):
    # Resolve the exact numbered version: later alias changes cannot alter the export.
    mv = client.get_model_version(MODEL_NAME, str(version))
    model = mlflow.sklearn.load_model(f"models:/{MODEL_NAME}/{mv.version}")
    folder = ROOT / "models" / release
    folder.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, folder / "pipeline.joblib", compress=3)
    meta = {"model_name": MODEL_NAME, "model_version": str(mv.version), "release": release,
```

```
            "run_id": mv.run_id, "model_uri": f"models:/{MODEL_NAME}/{mv.version}",
            "pipeline_sha256": sha256(folder / "pipeline.joblib"),
            "raw_sha256": dataset["raw_sha256"], "split_sha256": dataset["split_sha256"],
            "classes": {"0": "ham", "1": "spam"}, "input": {"texts": "1..100 nonblank
strings; each <=5000 characters"},
            "threshold": 0.5, "python": platform.python_version(),
            "scikit_learn": importlib.metadata.version("scikit-learn"),
            "algorithm": type(model.named_steps["classifier"]).__name__, "exported_at":
utc_now()}
    write_json(folder / "metadata.json", meta)
    return meta


def run(force=False):
    import os
    os.chdir(ROOT)
    frames, dataset = prepare_data()
    client, experiment_id = setup_tracking()
    summary_path = ROOT / "results/summary.json"
    if summary_path.exists() and not force:
        summary = read_json(summary_path)
        if summary["dataset"]["split_sha256"] != dataset["split_sha256"]:
            raise ValueError("Saved results use a different split")
        for item in summary["releases"].values():
            client.get_model_version(MODEL_NAME, item["model_version"])
            assert sha256(ROOT / "models" / item["release"] / "pipeline.joblib") ==
item["pipeline_sha256"]
        print("Используются сохранённые результаты. Для новых запусков: --force.")
        return summary
    results = ROOT / "results"
    results.mkdir(exist_ok=True)
    (results / "figures").mkdir(exist_ok=True)
    train, val = frames["train"], frames["validation"]
    records = []
    for order, (algorithm, parameter, value) in enumerate(CONFIGS):
        classifier = (MultinomialNB(alpha=value) if algorithm == "nb" else
                      LogisticRegression(C=value, max_iter=2000, solver="liblinear",
random_state=SEED))
        model = Pipeline([("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=20000,
                                                    sublinear_tf=True, min_df=1)),
                          ("classifier", classifier)])
        config_id = f"{algorithm}_{parameter}_{value:g}"
        with mlflow.start_run(run_name=config_id) as active:
            mlflow.log_params({"algorithm": type(classifier).__name__, parameter: value,
                               "seed": SEED, "train_rows": len(train), "validation_rows":
len(val),
                               "tfidf_ngram_range": "1,2", "tfidf_max_features": 20000,
                               "tfidf_sublinear_tf": True, "tfidf_min_df": 1,
                               "tfidf_lowercase": True, "tfidf_norm": "l2", "threshold":
0.5})
            mlflow.set_tags({"variant": "2", "purpose": "lab4", "config_id": config_id,
                             "dataset_sha256": dataset["raw_sha256"],
                             "split_sha256": dataset["split_sha256"],
                             "selection_metric": "val_f1_spam", "positive_class": "spam"})
            started = time.perf_counter()
            model.fit(train.text.tolist(), train.target)
            fit_seconds = time.perf_counter() - started
            pred = model.predict(val.text.tolist())
            scores = metrics(val.target, pred, "val")
            scores["fit_seconds"] = fit_seconds
            scores["vocabulary_size"] = len(model.named_steps["tfidf"].vocabulary_)
            mlflow.log_metrics(scores)
            for name in ("train", "validation"):
                mlflow.log_input(mlflow.data.from_pandas(frames[name][["text", "target"]],
                                                         source=SOURCE, targets="target"),
context=name)
            mlflow.log_artifact(str(ROOT / "data/splits.json"), artifact_path="data")
            mlflow.log_artifact(str(ROOT / "data/manifest.json"), artifact_path="data")
            for code in sorted((ROOT / "sms_mlops").glob("*.py")):
                mlflow.log_artifact(str(code), artifact_path="source/sms_mlops")
            for name in ("requirements.in", "requirements.txt", "requirements-
runtime.txt"):
                if (ROOT / name).exists():
                    mlflow.log_artifact(str(ROOT / name), artifact_path="source")
            fig, ax = plt.subplots(figsize=(5, 4))
```

```
            ConfusionMatrixDisplay.from_predictions(val.target, pred, labels=[0, 1],
                                                    display_labels=["ham", "spam"], ax=ax,
colorbar=False)
            ax.set_title(f"Validation: {config_id}")
            fig.tight_layout()
            mlflow.log_figure(fig, "validation_confusion_matrix.png")
            plt.close(fig)
            example = np.array(train.text.iloc[:2].tolist(), dtype=str)
            model_info = mlflow.sklearn.log_model(
                model, artifact_path="model", input_example=example,
                signature=infer_signature(example, model.predict(example)),
                pip_requirements=[line for line in (ROOT / "requirements-
runtime.txt").read_text().splitlines()
                                  if line and not line.startswith("#")])
            record = {"config_id": config_id, "algorithm": type(classifier).__name__,
                      "parameter": parameter, "value": value, "run_id": active.info.run_id,
                      "model_uri": model_info.model_uri, "order": order, **scores}
            records.append(record)
            print(config_id, active.info.run_id, scores, flush=True)
    # Fixed rule: highest validation F1, then precision, then first config in declared
order.
    ranked = sorted(records, key=lambda x: (-x["val_f1_spam"], -x["val_precision_spam"],
x["order"]))
    champion = ranked[0]
    alternative = next(row for row in ranked if row["algorithm"] != champion["algorithm"])
    pd.DataFrame(records).to_csv(results / "experiments.csv", index=False)
    releases = {}
    for release, selected in (("v1", alternative), ("v2", champion)):
        mv = mlflow.register_model(selected["model_uri"], MODEL_NAME)
        client.set_model_version_tag(MODEL_NAME, mv.version, "release", release)
        client.set_model_version_tag(MODEL_NAME, mv.version, "config_id",
selected["config_id"])
        client.update_model_version(MODEL_NAME, mv.version,
                                    description=f"{release}: {selected['config_id']};
selected using validation only")
        if release == "v2":
            client.set_registered_model_alias(MODEL_NAME, "champion", mv.version)
            resolved = client.get_model_version_by_alias(MODEL_NAME, "champion")
            assert resolved.version == mv.version
            version = resolved.version
        else:
            version = mv.version
        releases[release] = export_version(client, version, release, dataset)
    # Final test evaluates ONLY the selected frozen model; no tuning after this point.
    model = joblib.load(ROOT / "models/v2/pipeline.joblib")
    test = frames["test"]
    pred = model.predict(test.text.tolist())
    final = {**metrics(test.target, pred, "test"),
             "confusion_matrix": confusion_matrix(test.target, pred, labels=[0,
1]).tolist(),
             "classification_report": classification_report(test.target, pred,
target_names=["ham", "spam"],
                                                            output_dict=True,
zero_division=0),
             "rows": len(test), "run_id": champion["run_id"],
             "model_version": releases["v2"]["model_version"]}
    with mlflow.start_run(run_id=champion["run_id"]):
        mlflow.log_metrics({key: value for key, value in final.items() if
key.startswith("test_")})
        mlflow.log_dict(final, "final_test_metrics.json")
    write_json(results / "test_metrics.json", final)
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay.from_predictions(test.target, pred, display_labels=["ham",
"spam"], ax=ax, colorbar=False)
    ax.set_title(f"Final test: {champion['config_id']}")
    fig.tight_layout()
    fig.savefig(results / "figures/test_confusion_matrix.png", dpi=150)
    plt.close(fig)
    df = pd.DataFrame(records).set_index("config_id")
    ax = df[["val_f1_spam", "val_precision_spam", "val_recall_spam"]].plot.bar(figsize=(11,
5), ylim=(0, 1.05))
    ax.set_ylabel("Metric for spam")
    ax.set_xlabel("Experiment")
    ax.figure.tight_layout()
    ax.figure.savefig(results / "figures/experiments.png", dpi=150)
```

```
    plt.close(ax.figure)
    # Include both labels plus a validation SMS on which releases disagree, if present.
    samples = [val[val.label == label].text.iloc[0] for label in ("ham", "spam")]
    old_model = joblib.load(ROOT / "models/v1/pipeline.joblib")
    different = np.flatnonzero(old_model.predict(val.text.tolist()) !=
model.predict(val.text.tolist()))
    if len(different):
        samples.append(val.text.iloc[int(different[0])])
    write_json(ROOT / "examples/valid.json", {"texts": samples})
    for release in releases:
        exported = joblib.load(ROOT / "models" / release / "pipeline.joblib")
        write_json(ROOT / "examples" / f"expected_{release}.json",
                   {"predictions": ["spam" if p == 1 else "ham" for p in
exported.predict(samples)],
                    "probabilities_spam": exported.predict_proba(samples)[:, 1].tolist(),
                    "model_version": releases[release]["model_version"]})
    summary = {"created_at": utc_now(), "experiment_id": experiment_id, "dataset": dataset,
               "selection_rule": "max val_f1_spam; ties: max val_precision_spam, then
config order",
               "champion": champion, "alternative": alternative, "releases": releases,
"test": final,
               "environment": {"python": sys.version, "platform": platform.platform(),
                               "packages": {p: importlib.metadata.version(p) for p in
                                            ("mlflow", "scikit-learn", "numpy", "pandas",
"fastapi", "pydantic")}}}
    write_json(summary_path, summary)
    return summary


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--force", action="store_true", help="Create six NEW runs and two
NEW Registry versions")
    args = parser.parse_args()
    result = run(force=args.force)
    print("Champion:", result["champion"]["config_id"], "Registry",
result["releases"]["v2"]["model_version"])
```

sms_mlops/verify.py

```
"""The same API checks run against TestClient and a real Docker HTTP endpoint."""
import argparse
import time

import joblib
import numpy as np
import pandas as pd
from fastapi.testclient import TestClient

from .common import ROOT, read_json, sha256, utc_now, write_json
from .service import create_app

INVALID = [
    ("missing_texts", {}), ("empty_batch", {"texts": []}),
    ("empty_sms", {"texts": [""]}), ("blank_sms", {"texts": ["  \t\n"]}),
    ("number", {"texts": [42]}), ("boolean", {"texts": [True]}),
    ("null_sms", {"texts": [None]}), ("mixed_types", {"texts": ["Hello", 1]}),
    ("string_instead_of_batch", {"texts": "Hello"}),
    ("null_batch", {"texts": None}), ("wrong_top_level", ["Hello"]),
    ("extra_field", {"texts": ["Hello"], "unexpected": True}),
    ("oversized_batch", {"texts": ["Hello"] * 101}),
    ("oversized_sms", {"texts": ["x" * 5001]}),
]


def check_client(client, release):
    meta = read_json(ROOT / "models" / release / "metadata.json")
    model = joblib.load(ROOT / "models" / release / "pipeline.joblib")
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok", "model_version": meta["model_version"],
"release": release}
    api_meta = client.get("/metadata").json()
```

```
    assert api_meta["pipeline_sha256"] == meta["pipeline_sha256"]
    assert api_meta["run_id"] == meta["run_id"]
    assert client.get("/openapi.json").status_code == 200
    request = read_json(ROOT / "examples/valid.json")
    response = client.post("/predict", json=request)
    assert response.status_code == 200, response.text
    example = response.json()
    tests = []
    for name, payload in INVALID:
        bad = client.post("/predict", json=payload)
        assert bad.status_code == 422, (name, bad.status_code, bad.text)
        tests.append({"case": name, "expected_status": 422, "actual_status":
bad.status_code})
    bad = client.post("/predict", content="{broken", headers={"Content-Type":
"application/json"})
    assert bad.status_code == 422
    tests.append({"case": "malformed_json", "expected_status": 422, "actual_status":
bad.status_code})
    # All held-out SMS, not just one hand-picked request; no refitting happens here.
    data = pd.read_csv(ROOT / "data/clean.csv", index_col="row_id", keep_default_na=False)
    ids = read_json(ROOT / "data/splits.json")["test"]
    texts = data.loc[ids, "text"].tolist()
    texts.extend(["Привет, как дела?", "😀", "!@#$%", "Hello", "x" * 5000])
    # Check exact batch-size boundary as well as one-object requests.
    maximum_difference = 0.0
    for offset in range(0, len(texts), 100):
        batch = texts[offset:offset + 100]
        answer = client.post("/predict", json={"texts": batch})
        assert answer.status_code == 200, answer.text
        body = answer.json()
        expected = ["spam" if p == 1 else "ham" for p in model.predict(batch)]
        assert body["predictions"] == expected
        probabilities = model.predict_proba(batch)[:, 1]
        np.testing.assert_allclose(body["probabilities_spam"], probabilities, rtol=1e-10,
atol=1e-12)
        maximum_difference = max(maximum_difference,
                                 float(np.max(np.abs(np.asarray(body["probabilities_spam"])
- probabilities))))
        assert body["model_version"] == meta["model_version"] and body["release"] ==
release
        assert body["run_id"] == meta["run_id"]
    single = client.post("/predict", json={"texts": ["See you at six!"]})
    assert single.status_code == 200 and len(single.json()["predictions"]) == 1
    wall_ms, prediction_ms = [], []
    for _ in range(10):
        started = time.perf_counter()
        response = client.post("/predict", json=request)
        wall_ms.append((time.perf_counter() - started) * 1000)
        assert response.status_code == 200
        prediction_ms.append(response.json()["latency_ms"])
    return {"release": release, "model_version": meta["model_version"], "pipeline_sha256":
meta["pipeline_sha256"],
            "status": "passed", "parity_rows": len(texts), "max_probability_difference":
maximum_difference,
            "invalid_requests": tests, "example_request": request, "example_response":
example,
            "latency": {"requests": 10, "batch_size": len(request["texts"]),
                        "roundtrip_median_ms": float(np.median(wall_ms)),
                        "roundtrip_p95_ms": float(np.percentile(wall_ms, 95)),
                        "handler_median_ms": float(np.median(prediction_ms))}}


def run_local():
    reports = []
    # Starts a fresh app per release, exercising the startup loader on every switch.
    for release in ("v1", "v2", "v1"):
        with TestClient(create_app(ROOT / "models" / release)) as client:
            reports.append(check_client(client, release))
    assert reports[0]["example_response"]["predictions"] ==
reports[2]["example_response"]["predictions"]
    result = {"status": "passed", "transport": "FastAPI TestClient, in-process",
              "created_at": utc_now(), "sequence": reports,
              "note": "Local checks exercise API and exports. Docker results are recorded
separately."}
    write_json(ROOT / "results/local_checks.json", result)
```

```
    return result


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--url", help="Real running HTTP endpoint, e.g.
http://127.0.0.1:8000")
    parser.add_argument("--release", choices=["v1", "v2"], default="v2")
    args = parser.parse_args()
    if args.url:
        import httpx
        with httpx.Client(base_url=args.url, timeout=30, trust_env=False) as client:
            report = check_client(client, args.release)
        write_json(ROOT / "results/http_checks.json", report)
        print(report)
    else:
        result = run_local()
        print("PASS: API, invalid requests and export parity; lifecycle v1 → v2 → v1")
        for item in result["sequence"]:
            print(item["release"], item["model_version"], item["parity_rows"],
item["latency"])
```

tests/test_service.py

```
"""Startup integrity and export/API contract regressions."""
import json
import shutil

import pytest
from fastapi.testclient import TestClient

from sms_mlops.common import ROOT
from sms_mlops.service import create_app
from sms_mlops.verify import check_client


@pytest.mark.parametrize("release", ["v1", "v2"])
def test_export_and_api_contract(release):
    with TestClient(create_app(ROOT / "models" / release)) as client:
        report = check_client(client, release)
    assert report["status"] == "passed"


def test_corrupt_export_prevents_startup(tmp_path):
    for name in ("pipeline.joblib", "metadata.json"):
        shutil.copy(ROOT / "models/v2" / name, tmp_path / name)
    with (tmp_path / "pipeline.joblib").open("ab") as stream:
        stream.write(b"corrupted")
    with pytest.raises(RuntimeError, match="SHA-256 mismatch"):
        with TestClient(create_app(tmp_path)):
            pass
```

requirements.in

```
# Python 3.12; requirements.txt includes the resolved transitive dependencies.
mlflow==2.22.2
sqlalchemy==2.0.44
scikit-learn==1.6.1
numpy==2.2.6
pandas==2.2.3
scipy==1.15.3
fastapi==0.115.12
uvicorn==0.34.2
pydantic==2.11.5
httpx==0.28.1
matplotlib==3.10.3
nbconvert==7.16.6
nbformat==5.10.4
ipykernel==6.29.5
pytest==8.3.5
```

```
pypdf==5.5.0
```

requirements.txt

```
# Full verified Python 3.12 environment
alembic==1.20.1
annotated-types==0.8.0
anyio==4.16.0
asttokens==3.0.2
attrs==26.1.0
beautifulsoup4==4.15.0
bleach==6.4.0
blinker==1.9.0
cachetools==5.5.2
certifi==2026.10.23
cffi==2.1.1
charset-normalizer==3.5.2
click==8.5.1
cloudpickle==3.1.2
comm==0.2.3
contourpy==1.4.0
cryptography==50.0.2
cycler==0.12.1
databricks-sdk==0.164.0
debugpy==1.8.22
defusedxml==0.7.1
docker==7.2.0
executing==2.3.0
fastapi==0.115.12
fastjsonschema==2.22.3
Flask==3.1.3
fonttools==4.68.0
gitdb==4.0.12
GitPython==3.2.1
google-auth==2.62.1
graphene==3.4.3
graphql-core==3.2.14
graphql-relay==3.2.0
greenlet==3.5.6
gunicorn==23.0.0
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.20
importlib_metadata==8.9.0
iniconfig==2.3.1
ipykernel==6.29.5
ipython==9.18.1
ipython_pygments_lexers==1.1.1
itsdangerous==2.2.0
jedi==0.20.1
Jinja2==3.1.6
joblib==1.6.0
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
jupyter_client==8.10.0
jupyter_core==5.9.1
jupyterlab_pygments==0.3.0
kiwisolver==1.5.1
Mako==1.4.3
Markdown==3.11.3
MarkupSafe==3.0.4
matplotlib==3.10.3
matplotlib-inline==0.2.2
mistune==3.4.0
mlflow==2.22.2
mlflow-skinny==2.22.2
nbclient==0.11.0
nbconvert==7.16.6
nbformat==5.10.4
nest-asyncio==1.6.0
numpy==2.2.6
opentelemetry-api==1.45.2
```

```
opentelemetry-sdk==1.45.2
opentelemetry-semantic-conventions==0.66b2
packaging==24.2
pandas==2.2.3
pandocfilters==1.5.1
parso==0.8.7
pexpect==4.9.0
pillow==13.0.0
platformdirs==4.14.5
pluggy==1.7.0
prompt_toolkit==3.0.53
protobuf==6.33.6
psutil==7.2.2
ptyprocess==0.7.0
pure_eval==0.2.4
pyarrow==19.0.1
pyasn1==0.6.5
pyasn1_modules==0.4.2
pycparser==3.12
pydantic==2.11.5
pydantic_core==2.33.2
Pygments==2.21.0
pyparsing==3.3.3
pypdf==5.5.0
pytest==8.3.5
python-dateutil==2.9.0.post0
pytz==2026.6
PyYAML==6.0.3
pyzmq==27.3.0
referencing==0.37.0
requests==2.34.2
rpds-py==2026.9.1
scikit-learn==1.6.1
scipy==1.15.3
six==1.17.0
smmap==5.0.3
soupsieve==4.1
SQLAlchemy==2.0.44
sqlparse==0.6.1
stack-data==0.6.3
starlette==0.46.2
threadpoolctl==3.8.0
tinycss2==1.5.1
tornado==6.6.1
traitlets==5.16.1
typing-inspection==0.4.5
typing_extensions==4.16.0
tzdata==2026.6
urllib3==2.8.0
uvicorn==0.34.2
wcwidth==0.9.4
webencodings==0.6.1
Werkzeug==3.1.9
zipp==4.1.2
```

requirements-runtime.txt

```
# Runtime closure, resolved on Ubuntu 24.04 / Python 3.12
annotated-types==0.8.0
anyio==4.16.0
click==8.5.1
cloudpickle==3.1.2
fastapi==0.115.12
h11==0.16.0
idna==3.20
joblib==1.6.0
numpy==2.2.6
pydantic==2.11.5
pydantic_core==2.33.2
scikit-learn==1.6.1
scipy==1.15.3
starlette==0.46.2
threadpoolctl==3.8.0
```

```
typing_extensions==4.16.0
typing-inspection==0.4.5
uvicorn==0.34.2
```

Dockerfile

```
FROM python:3.12.10-slim-
bookworm@sha256:fd95fa221297a88e1cf49c55ec1828edd7c5a428187e67b5d1805692d11588db
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 MODEL_DIR=/app/model
WORKDIR /app
COPY requirements-runtime.txt .
RUN python -m pip install --no-cache-dir -r requirements-runtime.txt
COPY sms_mlops/__init__.py sms_mlops/service.py ./sms_mlops/
ARG MODEL_RELEASE=v1
COPY models/${MODEL_RELEASE}/ ./model/
RUN useradd --uid 10001 --create-home appuser
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=5s --timeout=3s --start-period=10s --retries=5 \
  CMD ["python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2)"]
CMD ["python", "-m", "uvicorn", "sms_mlops.service:app", "--host", "0.0.0.0", "--port",
"8000"]
```

compose.yaml

```
name: lab4-sms-spam
services:
  api:
    image: sms-spam:${MODEL_RELEASE:-v1}
    build:
      context: .
      args:
        MODEL_RELEASE: ${MODEL_RELEASE:-v1}
    ports:
      - "127.0.0.1:${API_PORT:-8000}:8000"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2)"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s
```

.dockerignore

```
**
!Dockerfile
!requirements-runtime.txt
!sms_mlops/
!sms_mlops/__init__.py
!sms_mlops/service.py
!models/
!models/v1/
!models/v1/pipeline.joblib
!models/v1/metadata.json
!models/v2/
!models/v2/pipeline.joblib
!models/v2/metadata.json
```

examples/expected_v1.json

```
{
  "predictions": [
```

```
    "ham",
    "ham",
    "spam"
  ],
  "probabilities_spam": [
    0.006182487177051617,
    0.4767839621766742,
    0.9178038431522817
  ],
  "model_version": "1"
}
```

examples/expected_v2.json

```
{
  "predictions": [
    "ham",
    "ham",
    "ham"
  ],
  "probabilities_spam": [
    0.0010209079134395204,
    0.1632287445930936,
    0.4945835957774673
  ],
  "model_version": "2"
}
```

examples/invalid_empty.json

```
{"texts": []}
```

examples/invalid_extra.json

```
{"texts": ["Hello"], "unexpected": true}
```

examples/invalid_type.json

```
{"texts": [42]}
```

examples/valid.json

```
{
  "texts": [
    "And how you will do that, princess? :)",
    "cmon babe, make me horny, *turn* me on! Txt me your fantasy now babe -) Im hot, sticky
and need you now. All replies cost £1.50. 2 cancel send STOP",
    "\"For the most sparkling shopping breaks from 45 per person; call 0121 2025050 or
visit www.shortbreaks.org.uk\""
  ]
}
```

Контрольные суммы для воспроизведения

SHA-256 неизменённого SMSSpamCollection:

```
7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d
```

SHA-256 data/splits.json:

c7c7f9c6bd2b5ab276809d4b7b134446449498cda7385b9e78e99e8fea279c5b

Проверенные результаты хранятся в results/summary.json, results/experiments.csv, results/test_metrics.json, results/local_checks.json и results/docker_checks.json. При переносе проекта необходимо сохранить mlflow-data целиком; python -m sms_mlops.relocate обновляет пути к артефактам в копии SQLite с предварительным резервным копированием.